

Addis Ababa University
College of Natural and Computational Science
Department of Physics

Computational Physics Application

(Phys 4624)

Ising Model in 1D Implementation

Name: Getabalew Manegerew

ID Number: UGR/6981/15

Instructor: Dr. Lemi D.

May 26, 2026

1 Introduction

In this project the Ising model in 1D is implemented. I specifically used the classical model which does not require considering the uncertainty arising from quantum systems. The essential idea of the Ising model is stated by its energy function in terms of specified spins at lattice points.

$$E = -J \sum_{i=1}^N s_i s_{i+1} \quad (1)$$

Where J is the spin coupling constant. Equation 1 is valid for a 1D periodically bounded N chain of spin systems. I also excluded the magnetic field term, since this project does not consider it.

It can also be shown that flipping the spin of the n th state corresponds to a change in energy and magnetization of, respectively,

$$\Delta E = 2J s_n (s_{n-1} + s_{n+1}) \quad (2)$$

$$\Delta M = 2s_n \quad (3)$$

In the 1D Ising model there is no phase transition at finite temperature. However, it is still possible to capture essential statistical mechanics ideas such as mean energy, magnetization, specific heat capacity, and so on. I compared the results of my simulation with the analytical results, which are found to be given as,

$$\langle E \rangle = -NJ \tanh(\beta J) \quad (4)$$

$$\langle M \rangle = \frac{e^{\beta J} \sinh(\beta H)}{\sqrt{e^{2\beta J} \sinh^2(\beta H) + e^{-2\beta J}}} \quad (5)$$

$$C = Nk_B(\beta J)^2 \operatorname{sech}^2(\beta J) \quad (6)$$

H is the magnetic field intensity and is 0 in our case, hence $\langle M \rangle = 0$.

2 Methodology

2.1 Algorithm of Implementation

The simulation follows the Metropolis-Hastings algorithm to sample the equilibrium distribution, in which the energy of the system is given by 1:

1. **Initialization:** Start with an initial spin state of N chains of spins $|S_0\rangle$.
2. **Picking a Lattice Point:** Generate a random integer n in the range $[1, N]$, which is the lattice point under discussion.
3. **Compute Changes:** Compute the change in energy and magnetization using 2 and 3, respectively.
4. **Probability Ratio:** Calculate the ratio of the probability of the system being in the new and old state according to the Boltzmann distribution ($e^{-\beta \Delta E}$ for $\Delta E > 0$).

5. Acceptance Criteria:

- If ratio > 1 or $\Delta E \leq 0$, accept the spin flip of the n th position. Then add ΔE and ΔM respectively to the previous values of energy and magnetization, and store them for further analysis.
 - If ratio < 1 , accept with probability ratio $= e^{-\beta\Delta E}$. This is done by picking a random number and comparing it with the ratio. In this case the energy and magnetization of the corresponding state remain the same as the previous state.
6. **Iteration:** Repeat for a total of $\approx N^2$ Monte Carlo (MC) steps, i.e. $N = 100$ as required.

2.2 Implementation Parameters

The simulation is performed using the following parameters as specified:

- **Temperature (β):** 10.0
- **Initial Configuration ($|S_0\rangle$):** Unbiased state of spins using random numbers
- **Interaction Coupling Constant (J):** 0.1
- **Number of Particles:** 150

3 Results

3.1 Statistical Moments

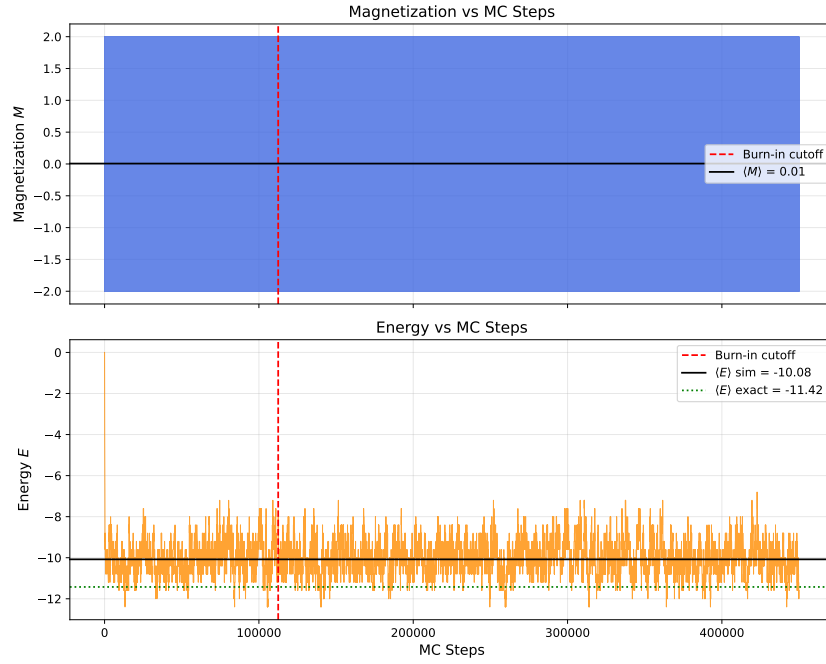
The calculated mean energy and mean magnetization are compared to the analytical expectations:

Metric	Simulated Value	Analytical Value
Mean Magnetization $\langle M \rangle$	-0.0077155	0
Mean Energy $\langle E \rangle$	-12.400540	-11.423912
Specific Heat Capacity $\langle C \rangle$	0.41534463	0.41997434

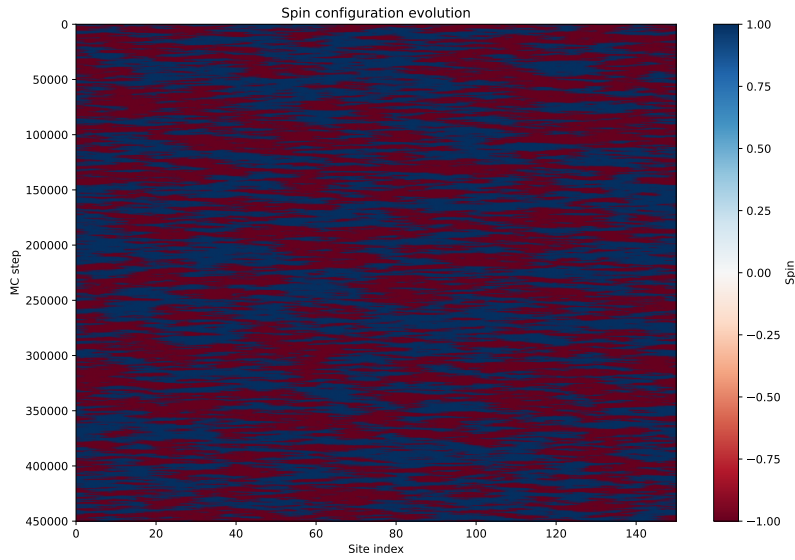
Table 1: Comparison of simulation statistics with exact physical values.

3.2 Visualizations

After running the MC cycles, the magnetization and energy of the given state were collected, which are displayed in figure [1a](#). Additionally, the heatmap of the state of each spin through computing time is shown in figure [1b](#).



(a) Magnetization and energy fluctuations as a function of Monte Carlo steps



(b) The heatmap of the spin state of each lattice as MC steps increase

Figure 1: These figures display how the system evolves as we progress in Monte Carlo steps

4 Analysis and Discussion

As can be inferred from table 1 and the graphical illustrations of 1, the implementation of the 1D Ising model achieved significant agreement with theoretical expectations. We see that the magnetization of the system varies less than the energy. The energy fluctuation is due to the sweeping effect and domain formations. This system is unstable without a magnetic field.

The agreement in specific heat capacity with the analytical result is very close. In Ising models, this is one of the best ways to check model implementations. In our case this indicates the effectiveness of the program.

However, this is a fictitious system. Real systems are not restricted to 1D and there are also long range interactions at a given time. One can introduce such effects for practical explorations.

5 Conclusion

In this project I simulated a 1D classical Ising model. Important parameters such as mean energy and magnetization were found to be in great agreement with analytical results. The energy of the system shows a noticeable variation in MC steps, which is attributed to domain formation.

This program implementation does not suffice for practical experiences. Real systems need higher dimensional simulation as well as consideration of long range interactions.

A Source Code

```
1 # -*- coding: utf-8 -*-
2 """
3 Created on Tue May 26 13:20:34 2026
4 @author: getsh
5 """
6 import numpy as np
7 import matplotlib.pyplot as plt
8
9 #defining simulation parameters
10 N=150; N_trials = 20*N**2
11 beta = 10; J=0.1
12
13 #initializing spin state, magnetization and energy arrays
14 Spin_state = np.zeros(N)
15 Magnetization = np.zeros(N_trials)
16 E = np.zeros(N_trials)
17
18 #defining boltzmann factor for probablity comparison
19 def boltz_factor(E):
20     return np.exp(-beta*E)
21
22 #Initializing unbaised spin state
23 for i in range(N):
24     r = np.random.uniform()
25     if (r<0.5):
26         Spin_state[i] = -1
```

```

27     else:
28         Spin_state[i] = 1
29
30 #computing initial magnetization
31 Magnetization[0] = sum(Spin_state)
32
33 # summing the energy of the state
34 Nm = N-1
35 for m in range(N):
36     E[0] += -J*Spin_state[m]*Spin_state[Nm]
37     Nm = m
38
39 # Store snapshots every N steps
40 snapshot_every = N
41 n_snapshots = N_trials // snapshot_every
42 snapshots = np.zeros((n_snapshots, N))
43
44 #main Monte Carlo loop
45 for j in range(N_trials):
46     #selecting a random spin site
47     n = int(N*np.random.uniform())
48
49     #storing snapshot of spin state every N steps
50     if j % snapshot_every == 0:
51         snapshots[j // snapshot_every] = Spin_state.copy()
52
53     # setting periodic boundary condition for nl and nr, left and
54     # right position of n
55     if (n==0):
56         nl = N - 1
57         nr = n + 1
58     elif (n==N - 1):
59         nl = n - 1
60         nr = 0
61     else:
62         nl = n - 1
63         nr = n + 1
64
65     dE = 2*J*Spin_state[n]*(Spin_state[nl] + Spin_state[nr]) #
66     #determining energy difference
67     w = boltz_factor(dE)
68     r = np.random.uniform()
69
70     if (dE<0 or w > r): # accepting the change
71         Spin_state[n] = - Spin_state[n] # flipping the nth spin
72         state
73         Magnetization[j] = Magnetization[j] + 2*Spin_state[n] #
74         #updating magnetization incrementally
75         E[j] = E[j-1] + dE # updating energy incrementally
76
77     else: # no change in the spin state
78         Magnetization[j] = Magnetization[j-1] # carrying forward
79         #previous magnetization
80         E[j] = E[j-1] # carrying forward previous energy
81
82 #plt.contour(Magnetization)
83
84 #setting burn-in cutoff and computing post burn-in averages

```

```

80 burnin = 10*N**2
81 E_mean = np.mean(E[burnin:])
82 E_exact = -N*J*np.tanh(beta*J) # analytical mean energy
83 M_mean = np.mean(Magnetization[burnin:])
84 C = beta**2*(np.mean(E[burnin:]**2) - np.mean(E[burnin:])**2)/N #
    specific heat from energy fluctuations
85 C_exact = (beta*J/np.cosh(beta*J))**2 # analytical specific heat
86 print(M_mean, E_mean, E_exact)
87
88 % Plot spacetime diagram
89 plt.figure(figsize=(12, 8))
90 plt.imshow(snapshots, aspect='auto', cmap='RdBu',
91           vmin=-1, vmax=1,
92           extent=[0, N, N_trials, 0])
93 plt.colorbar(label='Spin')
94 plt.xlabel('Site index')
95 plt.ylabel('MC step')
96 plt.title('Spin configuration evolution')
97
98 #setting up figure with two subplots sharing x axis
99 fig, axes = plt.subplots(2, 1, figsize=(10, 8), sharex=True)
100
101 % --- Magnetization plot ---
102 axes[0].plot(range(N_trials), Magnetization, color='royalblue',
103             linewidth=0.5, alpha=0.8)
104 axes[0].axvline(x=5*N**2, color='red', linestyle='--',
105                linewidth=1.5, label='Burn-in cutoff')
106 axes[0].axhline(y=np.mean(Magnetization[5*N**2:]), color='black',
107                linestyle='-', linewidth=1.5,
108                label=f'$\langle M \rangle$ = {np.mean(Magnetization
109                [5*N**2:]).2f}')
110 axes[0].set_ylabel('Magnetization $M$', fontsize=12)
111 axes[0].set_title('Magnetization vs MC Steps', fontsize=13)
112 axes[0].legend(fontsize=10)
113 axes[0].grid(True, alpha=0.3)
114
115 % --- Energy plot ---
116 axes[1].plot(range(N_trials), E, color='darkorange',
117             linewidth=0.5, alpha=0.8)
118 axes[1].axvline(x=5*N**2, color='red', linestyle='--',
119                linewidth=1.5, label='Burn-in cutoff')
120 axes[1].axhline(y=np.mean(E[5*N**2:]), color='black',
121                linestyle='-', linewidth=1.5,
122                label=f'$\langle E \rangle_{sim}$ = {np.mean(E[5*N**2:]).2f}')
123 axes[1].axhline(y=E_exact, color='green', linestyle=':',
124                linewidth=1.5, label=f'$\langle E \rangle_{exact}$ = {
125                E_exact:.2f}')
126 axes[1].set_ylabel('Energy $E$', fontsize=12)
127 axes[1].set_xlabel('MC Steps', fontsize=12)
128 axes[1].set_title('Energy vs MC Steps', fontsize=13)
129 axes[1].legend(fontsize=10)
130 axes[1].grid(True, alpha=0.3)

```

Listing 1: Python implementation of the Metropolis Algorithm for Boltzmann Sampling